Advanced Caching Architecture

HybridCache & The L3 SQL Server Strategy

A Senior Developer's Guide to Multi-Tiered Application Caching

Part 1: The HybridCache Masterclass (.NET 9+)

Before HybridCache, developers had to manually orchestrate the dance between L1 (In-Memory RAM) and L2 (Distributed Redis). This led to massive boilerplate code, manual JSON serialization, and the dreaded **Cache Stampede**. HybridCache solves all of this natively.

How It Works Under the Hood

- **L1 First:** It checks local `IMemoryCache`. If hit, it returns instantly (nanoseconds).
- **L2 Fallback:** If L1 misses, it checks `IDistributedCache` (Redis). If hit, it deserializes it, updates L1, and returns.
- **Locking (Stampede Protection):** If L1 and L2 both miss, it executes your database query. Crucially, if 1000 requests hit simultaneously, it **locks the execution**. Only 1 request hits SQL; the other 999 wait and receive the cached result the moment the 1st request finishes.

Implementation Details

```
// Program.cs
builder.Services.AddStackExchangeRedisCache(opt => ...);
#pragma warning disable EXTEXP0018
builder.Services.AddHybridCache(); // Automatically wires to Redis if registered

// Service Implementation
public async Task<Product> GetProductAsync(int id, CancellationToken ct)
{
    return await _hybridCache.GetOrCreateAsync(
        $"product_{id}",
        async token => await _db.Products.FindAsync([id], token),
        cancellationToken: ct
    );
}
```

> **CRITICAL CAUTIONS FOR HYBRIDCACHE:**
> 1. **Serialization Requirements:** Since HybridCache writes to L2, your objects MUST be serializable. No circular references (e.g., Entity Framework Navigation Properties referencing each other).
> 2. **Immutable Data:** L1 returns a direct memory reference. If you fetch a cached Product and modify it (e.g., `product.Price = 50;`) WITHOUT saving, you just mutated the cache for all other users! Always treat cached objects as read-only.

Part 2: The L3 Cache (SQL Server)

In web architecture, we classify cache layers by speed and distance from the CPU. But what happens when Redis (L2) is too expensive, or your cache data is so massive it exceeds your Redis RAM limit?

Enter the **L3 Cache: Using SQL Server as a Distributed Cache**.

> **The Tier System**
>
> - **L1 (In-Memory):** Lives in the App Server's RAM. Speed: Nanoseconds. Data lost on restart.
> - **L2 (Redis):** Lives in a separate RAM Server. Speed: ~1-3 Milliseconds. High cost for large RAM.
> - **L3 (SQL Server Cache):** Lives on Disk in a Database. Speed: ~10-20 Milliseconds. Slower than Redis, but significantly faster than running complex multi-table JOINs, and allows terabytes of cheap cache storage.

When and Why to use SQL Server as L3 Cache

Scenario	Why L3 (SQL) is Better than L2 (Redis) here
Massive Datasets	RAM is expensive. Storing 100GB of JSON in Redis costs hundreds of dollars a month. Storing 100GB in a SQL table costs pennies.
Strict Infrastructure	Many enterprise/government policies strictly limit the introduction of new infrastructure (like Docker/Redis). But they almost always have SQL Server approved and running.
Persistent Cache	Redis drops data if the container crashes (unless complex persistence is configured). SQL Server is strictly ACID compliant. The cache will survive a total data center reboot.

Implementation of L3 Cache

ASP.NET Core has a built-in provider just for this. First, you run a CLI tool to create the special Cache Table in your database:

```
dotnet tool install --global dotnet-sql-cache
dotnet sql-cache create "Data Source=...;Initial Catalog=DB" dbo CacheTable
```

Then, you wire it up in `Program.cs`. Notice how it uses the exact same `IDistributedCache` interface as Redis!

```
// Program.cs
builder.Services.AddDistributedSqlServerCache(options =>
{
    options.ConnectionString = builder.Configuration.GetConnectionString("Default");
    options.SchemaName = "dbo";
    options.TableName = "CacheTable";
});
```

The Ultimate Architecture: HybridCache + L3

You can actually combine these! If you register `AddDistributedSqlServerCache` instead of Redis, and then register `AddHybridCache`, your architecture becomes:

L1 (RAM for speed) -> falls back to -> L3 (SQL Server Cache for massive cheap storage) -> falls back to -> Heavy SQL JOIN queries.